UNIT 3 · TOPIC 3.1

Network-Level Exploits

Assignment, Answer Key

Teacher answer key, 41 points total.

Practice Questions

3.1.a · TCP/IP fundamentals and network topology flaws

1. Which TCP/IP layer is responsible for routing packets between networks using IP addresses? (1 pt)

- ○ A. Application layer
- ○ B. Transport layer
- **● C. Internet layer**
- ○ D. Network access layer

Why: The internet layer (IP) assigns addresses and routes packets between networks. The transport layer manages ports and reliability; the network access layer handles local delivery using MAC addresses.

2. Which of the following are structural weaknesses built into TCP/IP that attackers exploit? Select ALL that apply. (3 pts)

- **■ A. IP addresses are not authenticated**
- **■ B. DNS caches can accept forged replies**
- □ C. TCP uses encrypted sequence numbers by default
- **■ D. ARP operates on trust with no authentication**

Why: IP has no source-address verification (spoofing), DNS caches forgeable replies (poisoning), and ARP trusts any reply broadcast (ARP spoofing). TCP sequence numbers are NOT encrypted by default; that is a weakness, not a safeguard.

3. An attacker floods a switch with thousands of fake MAC addresses until it begins broadcasting all frames to every port. What attack is this? (1 pt)

- **● A. MAC flooding**
- ○ B. IP spoofing
- ○ C. ARP poisoning
- ○ D. DNS spoofing

Why: MAC flooding exhausts the switch's CAM table, forcing it to broadcast like a hub, enabling passive sniffing of all frames.

4. What network design practice limits how far an attacker can move laterally after compromising a single internal host? (1 pt)

Acceptable answers: segmentation; network segmentation; vlan; subnetting; subnet

Why: Network segmentation (VLANs, subnets with controlled routing) contains a breach to one zone, preventing a compromised host from reaching all others.

5. During a TCP three-way handshake, what is the correct order of messages? (1 pt)

- ○ A. ACK, SYN, SYN-ACK
- ○ B. SYN, ACK, SYN-ACK
- ○ C. SYN-ACK, SYN, ACK
- **● D. SYN, SYN-ACK, ACK**

Why: The client sends SYN, the server replies SYN-ACK, the client completes with ACK. Early implementations used predictable sequence numbers, enabling session hijacking.

6. Which layer of the TCP/IP stack does IP spoofing primarily target? (1 pt)

○ A. Application layer

○ B. Transport layer

● **C. Internet layer**

○ D. Network access layer

Why: IP spoofing forges the source address field in the IP header, which lives at the internet layer. Defenses (ingress filtering) also operate at this layer.

3.1.b · DNS attacks (spoofing, cache poisoning)

7. Why can an attacker forge a DNS response over UDP more easily than they could tamper with a TCP connection? (1 pt)

○ A. UDP packets are smaller, so a forged reply fits in fewer bytes

○ B. UDP has no checksum, so altered replies are not noticed

○ C. DNS servers do not log UDP traffic, so forgeries leave no trace

● **D. UDP is connectionless, so there is no handshake the resolver can use to verify the reply's origin**

Why: TCP's handshake proves both sides can receive each other's packets before data flows. UDP has no handshake: the resolver sends one packet and accepts the first reply that matches the transaction ID, port and question. An attacker who guesses those can win the race. UDP does have a checksum, but it only catches accidental damage.

8. Which THREE fields must a forged DNS reply match for a resolver to accept it as the answer to an outstanding query? (1 pt)

○ A. Source IP, TTL and record type

○ B. Transaction ID only, since it is unique per query

● **C. Destination port, query domain name, transaction ID**

○ D. Transaction ID, source MAC address and DNSSEC signature

Why: A resolver accepts a reply if it arrives on the port the query used, carries the same 16-bit transaction ID, and answers the same question (domain name). The transaction ID alone has only 65,536 values, which is why source-port randomization (adding about 16 more bits to guess) was the urgent fix after Kaminsky. MAC addresses do not survive across routers, and most resolvers did not check DNSSEC.

9. Which of the following are effective defenses against DNS cache poisoning? Select ALL that apply. (2 pts)

■ **A. Source port randomization**

■ **B. DNSSEC cryptographic signatures**

□ C. Hiding the resolver's IP address

□ D. Increasing the TTL of all records to one week

Why: Source port randomization widens the guessing space an attacker must brute-force, and DNSSEC lets a resolver cryptographically verify a reply even if an attacker wins the UDP race. Hiding the resolver's IP and raising TTLs do not address the underlying spoofing vulnerability.

10. In the Kaminsky attack, why does the attacker query unique, non-existent subdomains (e.g. 1.bank.com, 2.bank.com) instead of the base domain? (1 pt)

○ A. It is required by the DNS protocol specification

○ B. It bypasses DNSSEC entirely

● **C. It forces the resolver to issue a fresh query to the authoritative server every time, since there is no cached answer, giving the attacker repeated chances to win the race**

○ D. It reduces the number of packets the attacker needs to send

Why: A unique subdomain guarantees a cache miss, forcing a new query-and-race cycle each time, which multiplies the attacker's opportunities to guess the transaction ID before the real reply arrives.

11. Name the DNS extension that adds cryptographic signatures so a resolver can verify a reply even if an attacker wins the UDP timing race. (1 pt)

Acceptable answers: DNSSEC; domain name system security extensions

Why: DNSSEC signs DNS records with a key hierarchy rooted at the DNS root zone, letting resolvers reject unsigned or incorrectly signed forged replies.

12. A resolver's outstanding query has tx_id=500 and source port=9000. A reply arrives with tx_id=500, port=9000, but for a different query name than was asked. What should the resolver do? (1 pt)

○ A. Cache it anyway since the tx_id matches

● **B. Reject the reply as a mismatch**

○ C. Cache it anyway since the port matches

○ D. Forward it to the authoritative server for verification

Why: All three fields (tx_id, port, AND query name) must match; a mismatched query name means this reply does not answer the outstanding query and must be rejected.

3.1.c · Man-in-the-middle and ARP spoofing

13. Why does ARP allow spoofing so easily? (1 pt)

○ A. ARP replies are encrypted, so the switch cannot inspect them

○ B. ARP only runs when a device first joins, so stale entries are never corrected

○ C. Only the gateway is allowed to send ARP replies, and it can be impersonated

● **D. ARP has no built-in authentication, so any device can send a reply and it will be trusted**

Why: ARP was designed for trusted local networks: a host accepts any reply saying "IP x is at MAC y", even one it never asked for, and updates its cache. Nothing proves the sender owns that IP. ARP is not encrypted, any host can reply, and caches update constantly, which is exactly what spoofing exploits.

14. An attacker wants to intercept all of a victim's internet-bound traffic on a LAN using ARP spoofing. Which two devices' ARP tables does the attacker typically poison? (1 pt)

○ A. Two random unrelated hosts on the network

● **B. The victim's and the default gateway's**

○ C. Only the DNS server's

○ D. Only the victim's web browser cache

Why: To intercept traffic between the victim and the internet, the attacker poisons the victim's ARP table (to point 'gateway' at the attacker) and the gateway's ARP table (to point 'victim' at the attacker), placing itself in the middle of that path.

15. Which of the following are effective technical controls against ARP spoofing? Select ALL that apply. (2 pts)

■ **A. Dynamic ARP Inspection (DAI) on switches**

■ **B. Static ARP entries for critical hosts like the gateway**

□ C. Renaming the Wi-Fi network's SSID

■ **D. Network segmentation with VLANs**

Why: DAI validates ARP against a trusted binding table, static entries prevent the OS from accepting a forged update for a specific IP, and VLAN segmentation limits which hosts share a broadcast domain (and thus can spoof each other). Renaming an SSID has no effect on ARP spoofing.

16. Does HTTPS fully prevent damage from a successful ARP-spoofing man-in-the-middle position? (1 pt)

○ A. Yes, HTTPS makes MITM positioning impossible

○ B. No, HTTPS provides no protection at all against MITM

● C. No, HTTPS protects the confidentiality/integrity of the payload, but the attacker can still see which servers are contacted and can attempt downgrade or certificate-warning attacks

○ D. Yes, once ARP is poisoned, HTTPS automatically re-routes traffic safely

Why: HTTPS does not stop the attacker from occupying the MITM position via ARP spoofing; it limits what the attacker can do with that position (cannot read/alter the encrypted payload without detection), but metadata like destination server and connection timing remain visible, and downgrade/certificate attacks are still possible against unwary users.

17. Name the switch-level feature that validates ARP packets against a trusted IP-to-MAC binding table and drops forged ones. (1 pt)

Acceptable answers: dynamic arp inspection; dai

Why: Dynamic ARP Inspection (DAI) uses a trusted binding table (often built from DHCP snooping records) to validate and drop ARP packets that don't match a legitimate binding.

18. An ARP-watch tool observes IP 192.168.1.1 first bound to MAC aa:11, then 40 seconds later sees a reply claiming 192.168.1.1 is at MAC bb:22. How should this be classified? (1 pt)

● A. SPOOFED, a different MAC now claims an IP that was already bound to a different MAC

○ B. LEGIT, ARP bindings change routinely

○ C. Cannot be determined without a packet capture

○ D. LEGIT, only DNS bindings can be spoofed, not ARP

Why: A second, different MAC claiming an already-bound IP is exactly the ARP spoofing signature that arpwatch-style tools flag.

3.1.d · Packet capture and traffic sniffing

19. What NIC setting must a capture tool typically use to see traffic not addressed to its own MAC address? (1 pt)

○ A. DHCP mode

○ B. Airplane mode

● C. Promiscuous mode

○ D. Bridge mode

Why: Promiscuous mode tells the network stack to accept and process every frame the NIC physically receives, not just frames addressed to its own MAC, necessary for a capture tool to see traffic beyond its own.

20. What is the difference between a capture filter and a display filter in a tool like Wireshark? (1 pt)

○ A. A capture filter hides packets on screen, while a display filter deletes them from the file

● B. A capture filter determines what is recorded at all; a display filter narrows the view of what was already captured

○ C. Display filters run on the network card, while capture filters run in Wireshark

○ D. Capture filters can match payload text, while display filters can only match addresses

Why: Capture filters decide what gets saved in the first place (anything filtered out is gone for good), so they keep files small but risk missing evidence. Display filters only hide packets from view, and you can change them at any time without losing data. That is why analysts often capture broadly and filter the display.

21. Which of the following protocols send credentials in plaintext, making them fully readable in a packet capture? Select ALL that apply. (2 pts)

■ **A. Telnet**

□ B. HTTPS

■ **C. FTP (control channel)**

□ D. SSH

Why: Telnet and FTP's control channel send authentication data in cleartext. HTTPS and SSH both encrypt the connection, so credentials are not readable in a capture.

22. On a fully switched (non-hub) network, which technique lets an attacker still see another host's traffic without physical access to a SPAN port? (1 pt)

○ A. Renaming their hostname

○ B. Simply plugging into any switch port

○ C. Changing their own IP address

● **D. ARP spoofing to redirect the victim's traffic through the attacker's machine**

Why: A switch normally isolates traffic to the correct destination port; ARP spoofing (3.1.c) tricks two hosts into routing traffic through the attacker, restoring visibility that switching would otherwise prevent.

23. Name the legitimate switch feature, intended for monitoring tools, that copies traffic from one or more ports to a monitoring port, and which becomes a risk if granted to the wrong party. (1 pt)

Acceptable answers: port mirroring; span port; span; mirror port

Why: Port mirroring (a SPAN port) duplicates traffic to a monitoring port for legitimate analysis; if misconfigured or given to an unauthorized party, it enables unauthorized sniffing.

24. A capture log entry shows TCP port 443 traffic to a banking site with payload described only as encrypted binary data. What should an analyst conclude? (1 pt)

● **A. No credential can be extracted from this capture because TLS encrypts the payload**

○ B. The credential can be recovered by converting the binary payload to text

○ C. The capture proves the site is safe, because the traffic is encrypted

○ D. The analyst should look at the TCP header, where TLS stores the username

Why: TLS encrypts the application data, so the password is unreadable without the session keys. The capture still shows metadata (IP addresses, ports, the server name in the handshake, timing and sizes). Encryption says nothing about whether the site itself is legitimate; a phishing site can use HTTPS too.

Coding Assignments (Python 3)

3.1.a · TCP/IP fundamentals and network topology flaws

3 pts · auto-graded on the site

Coding problem 3.1.a: TCP/IP fundamentals and network topology flaws

Skill: Analyze Risk · **Maps to:** 3.1.A · **Language:** Python 3 (pytest)

Problem

A network monitoring tool has captured a log of IP packets on a corporate subnet. Each log entry records the packet's source IP, destination IP, and an expected source subnet (CIDR notation). Your job is to implement a spoofing detector: given a list of packet records, return the zero-based indices of packets whose source IP address falls outside the expected source subnet, which is a strong indicator of IP spoofing.

A packet is considered suspicious (spoofed) when `src_ip` does not belong to `expected_subnet`. Only IPv4 addresses are present. Subnets are given as `"A.B.C.D/N"` CIDR strings.

Function contract

Implement `solve(packets: list[dict]) -> list[int]` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

A list of dicts. Each dict has three string keys:

- `"src_ip"`: e.g. `"10.0.1.55"`
- `"dst_ip"`: e.g. `"10.0.1.1"`
- `"expected_subnet"`: e.g. `"10.0.1.0/24"`

Output

A sorted list of zero-based integer indices of the suspicious (spoofed) packets. Empty list if none are suspicious.

Examples

```
[{"src_ip": "10.0.1.55",  "dst_ip": "10.0.1.1", "expected_subnet": "10.0.1.0/24"},
 {"src_ip": "192.168.5.9","dst_ip": "10.0.1.1", "expected_subnet": "10.0.1.0/24"},
 {"src_ip": "10.0.1.200", "dst_ip": "10.0.1.1", "expected_subnet": "10.0.1.0/24"}]
-> [1]          # packet at index 1 has src_ip outside 10.0.1.0/24

[]  ->  []      # empty log: nothing to flag
```

Constraints

- Deterministic, offline, standard library only (`ipaddress` module is standard library).
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
import ipaddress
from typing import Any

def solve(data: Any) -> list[int]:
    if not isinstance(data, list):
        return []

    suspicious = []
    for idx, packet in enumerate(data):
        if not isinstance(packet, dict):
            continue
        src_ip = packet.get("src_ip")
        expected_subnet = packet.get("expected_subnet")
        if not src_ip or not expected_subnet:
            continue
        try:
            ip = ipaddress.ip_address(src_ip)
            net = ipaddress.ip_network(expected_subnet, strict=False)
            if ip not in net:
                suspicious.append(idx)
```

```
        except ValueError:
            suspicious.append(idx)

    return sorted(suspicious)
```

3.1.b · DNS attacks (spoofing, cache poisoning)

3 pts · auto-graded on the site

Coding problem 3.1.b: DNS resolver race classifier

Skill: Analyze Risk, Detect Attacks · **Maps to:** 3.1.A, 3.1.B · **Language:** Python 3 (pytest)

Problem

You are building triage logic for a resolver reply log at Crestfield Analytics. For each outstanding query, the log records the replies that arrived, and, for each reply, who sent it (`"authoritative"` or `"attacker"`), and the `tx_id`, `port`, and `qname` it carried. Classify each query's outcome:

- **`"REJECT"`**: no reply is a valid candidate (none match the query's `tx_id`, `port`, AND `qname` all three), so nothing is cached. This also covers the case of zero replies.
- **`"WIN"`**: among the valid candidates (replies matching all three fields), the one with the smallest `arrival` number (i.e. the first to arrive) has `"from": "authoritative"`.
- **`"POISON"`**: among the valid candidates, the first-arriving one has `"from": "attacker"`.

Function contract

Implement `solve(queries)` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `queries`: `list[dict]`, each with: - `"qname"`: `str`, the domain name queried - `"tx_id"`: `int`, the transaction ID the resolver sent - `"port"`: `int`, the source port the resolver sent the query from - `"replies"`: `list[dict]`, each with `"from"` (`"authoritative"` or `"attacker"`), `"tx_id"` (`int`), `"port"` (`int`), `"qname"` (`str`), `"arrival"` (`int`, lower = earlier)

Output

A `list[str]` of classifications, **one per input query, in the same order as the input**. Each entry is one of `"WIN"`, `"POISON"`, `"REJECT"`.

Examples

```
queries = [
    {
        "qname": "bank.example", "tx_id": 48291, "port": 33412,
        "replies": [
            {"from": "attacker", "tx_id": 48291, "port": 33412, "qname": "bank.example", "arrival": 1},
            {"from": "authoritative", "tx_id": 48291, "port": 33412, "qname": "bank.example", "arrival": 2},
        ],
    },
]
solve(queries)  ->  ["POISON"]

queries2 = [
    {
        "qname": "shop.example", "tx_id": 11020, "port": 40500,
        "replies": [
            {"from": "attacker", "tx_id": 99999, "port": 40500, "qname": "shop.example", "arrival": 1},
```

```
            {"from": "authoritative", "tx_id": 11020, "port": 40500, "qname": "shop.example", "arrival": 2},
        ],
    },
]
solve(queries2)  ->  ["WIN"]
```

Constraints

- Deterministic, offline, standard library only.
- A reply is a "valid candidate" only if `reply["tx_id"] == query["tx_id"]` AND `reply["port"] == query["port"]` AND `reply["qname"] == query["qname"]`.
- Among valid candidates, only the one with the minimum `arrival` value matters (test data never has ties on the minimum `arrival`).
- Replies may be given in any order in the list; use the `arrival` field, not list position, to determine which arrived first.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
from typing import List, Dict


def solve(queries: List[Dict]) -> List[str]:
    result = []
    for q in queries:
        candidates = [
            r for r in q["replies"]
            if r["tx_id"] == q["tx_id"] and r["port"] == q["port"] and r["qname"] == q["qname"]
        ]
        if not candidates:
            result.append("REJECT")
            continue
        winner = min(candidates, key=lambda r: r["arrival"])
        result.append("WIN" if winner["from"] == "authoritative" else "POISON")
    return result
```

3.1.c · Man-in-the-middle and ARP spoofing

3 pts · auto-graded on the site

Coding problem 3.1.c: ARP spoofing detector

Skill: Analyze Risk · **Maps to:** 3.1.B · **Language:** Python 3 (pytest)

Problem

You are building an arpwatch-style detector for Crestfield Analytics. Given a timestamp-ordered log of observed ARP replies (each mapping an IP to a MAC), identify which IP addresses show the ARP spoofing signature: the **first** reply seen for an IP establishes its trusted MAC binding. Any **later** reply for the same IP with a **different** MAC is a spoofing indicator. A later reply with the *same* MAC as the trusted binding is just a normal refresh and is not flagged.

Function contract

Implement `solve(replies)` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `replies`: `list[dict]`, each with: - `"ts"`: `int`, Unix timestamp (seconds) the reply was observed - `"ip"`: `str`, the IP address claimed in the reply - `"mac"`: `str`, the MAC address claimed for that IP

Output

A `list[str]`, the distinct IP addresses that show the spoofing signature (at least one reply with a MAC different from that IP's first-seen MAC), **sorted alphabetically**, with no duplicates.

Examples

```
replies = [
    {"ts": 100, "ip": "192.168.1.1", "mac": "aa:11:22:33:44:01"},
    {"ts": 130, "ip": "192.168.1.1", "mac": "aa:11:22:33:44:01"},
    {"ts": 145, "ip": "192.168.1.1", "mac": "de:ad:be:ef:00:99"},
]
solve(replies)  ->  ["192.168.1.1"]

replies2 = [
    {"ts": 100, "ip": "192.168.1.50", "mac": "11:22:33:44:55:66"},
]
solve(replies2)  ->  []
```

Constraints

- Deterministic, offline, standard library only.
- `replies` may be given in any timestamp order in the list; use the `ts` field (not list position) to determine which reply for a given IP came first (the "trusted" binding).
- Test data never has two replies for the same IP with an identical minimum `ts`.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
from typing import List, Dict


def solve(replies: List[Dict]) -> List[str]:
    by_ip = {}
    for r in replies:
        by_ip.setdefault(r["ip"], []).append(r)

    flagged = set()
    for ip, rs in by_ip.items():
        trusted = min(rs, key=lambda r: r["ts"])
        for r in rs:
            if r["mac"] != trusted["mac"]:
                flagged.add(ip)
                break

    return sorted(flagged)
```

3.1.d · Packet capture and traffic sniffing

3 pts · auto-graded on the site

Coding problem 3.1.d: Packet capture credential-leak detector

Skill: Detect Attacks · **Maps to:** 3.1.B, 3.1.C · **Language:** Python 3 (pytest)

Problem

You are triaging a captured packet log at Crestfield Analytics. For each entry, determine whether it exposes a plaintext credential ("LEAK") or not ("SAFE"):

- **Encrypted ports are always "SAFE"**: regardless of payload content: port 443 (HTTPS) and port 22 (SSH) encrypt their payload, so nothing in the payload text can be treated as an actual leaked credential.
- **Cleartext-capable ports** are 80 (HTTP), 23 (Telnet), and 21 (FTP). An entry on one of these ports is **"LEAK"** if its payload contains any credential indicator string (case-sensitive substring match), otherwise it is **"SAFE"**.
- Credential indicator strings to check for (substring match anywhere in the payload): `"password="`, `"Password:"`, `"PASS "`.
- Any other port not listed above is treated as `"SAFE"` (out of scope for this detector).

Function contract

Implement `solve(packets)` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `packets`: `list[dict]`, each with: - `"port"`: `int`, destination port - `"payload"`: `str`, the captured payload text

Output

A `list[str]` of classifications, **one per input packet, in the same order as the input**. Each entry is one of `"LEAK"`, `"SAFE"`.

Examples

```
packets = [
    {"port": 80, "payload": "POST /login HTTP/1.1\r\n\r\nuser=alice&password=Pass123!"},
    {"port": 443, "payload": "\\x16\\x03\\x03 encrypted application data"},
]
solve(packets)  ->  ["LEAK", "SAFE"]

packets2 = [
    {"port": 23, "payload": "login: admin\r\nPassword: R0uterAdmin!"},
    {"port": 80, "payload": "GET /index.html HTTP/1.1"},
]
solve(packets2)  ->  ["LEAK", "SAFE"]
```

Constraints

- Deterministic, offline, standard library only.
- Port check first: if `port` is 443 or 22, the result is always `"SAFE"`.
- Otherwise, if `port` is 80, 23, or 21: check for any of `"password="`, `"Password:"`, `"PASS "` as a substring of `payload`; if found, `"LEAK"`, else `"SAFE"`.
- Any other port: always `"SAFE"`.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
from typing import List, Dict

_ENCRYPTED_PORTS = {443, 22}
_CLEARTEXT_PORTS = {80, 23, 21}
_INDICATORS = ("password=", "Password:", "PASS ")


def solve(packets: List[Dict]) -> List[str]:
    result = []
    for p in packets:
        port = p["port"]
        if port in _ENCRYPTED_PORTS:
            result.append("SAFE")
        elif port in _CLEARTEXT_PORTS:
            payload = p["payload"]
            leaked = any(ind in payload for ind in _INDICATORS)
            result.append("LEAK" if leaked else "SAFE")
        else:
            result.append("SAFE")
    return result
```